In [4]:
```
import pandas as pd
import numpy as np
```

In [6]:
```
df = pd.read_csv("house_prices_practice.csv")
df.head(10)
```

Out[6]:

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	BedroomAbvGr	LotArea	SalePrice
0	1	7	1560	0	1658	1969	2	1	8059	177106
1	2	4	2827	2	1319	2012	3	4	13530	301044
2	3	8	3920	0	841	2010	1	4	9010	360609
3	4	5	3044	0	1058	1998	0	4	13207	240556
4	5	7	801	1	2428	2020	0	1	9117	193656
5	6	10	680	1	2480	1950	2	1	13081	213952
6	7	3	1106	2	1096	1962	1	4	11153	145539
7	8	7	3670	1	437	2000	2	5	4989	350830
8	9	8	1199	2	1653	2005	1	5	14040	275955
9	10	5	1492	1	2175	2011	1	2	11218	211664

Initial Data Analysis

In [7]:
```
df.shape
```

Out[7]:
```
(300, 10)
```

In [8]:
```
df.dtypes
```

Out[8]:
```
Id              int64
OverallQual     int64
GrLivArea       int64
GarageCars      int64
TotalBsmtSF     int64
YearBuilt       int64
FullBath        int64
BedroomAbvGr    int64
LotArea         int64
SalePrice       int64
dtype: object
```

In [9]:
```
df.isnull().sum()
```

Out[9]:
```
Id              0
OverallQual     0
GrLivArea       0
GarageCars      0
TotalBsmtSF     0
YearBuilt       0
FullBath        0
BedroomAbvGr    0
LotArea         0
SalePrice       0
dtype: int64
```

In [10]:
```
df.duplicated().sum()
```

Out[10]:
```
np.int64(0)
```

In [11]:
```
df.describe()
```

Out[11]:

	Id	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	BedroomAbvGr	LotArea	SalePrice
count	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000	300.000000
mean	150.500000	5.326667	2307.386667	1.330000	1468.796667	1986.163333	1.523333	2.926667	8969.453333	252262.903333
std	86.746758	2.873001	1042.561303	1.109898	672.333705	21.377089	1.131543	1.456604	3753.531132	74998.055214
min	1.000000	1.000000	504.000000	0.000000	303.000000	1950.000000	0.000000	1.000000	2009.000000	82494.000000
25%	75.750000	3.000000	1392.250000	0.000000	903.000000	1967.000000	0.000000	2.000000	5996.250000	190355.250000
50%	150.500000	5.000000	2265.500000	1.000000	1502.000000	1986.000000	2.000000	3.000000	9031.000000	251292.500000
75%	225.250000	8.000000	3306.500000	2.000000	2129.500000	2004.250000	3.000000	4.000000	12316.000000	307105.000000
max	300.000000	10.000000	3998.000000	3.000000	2492.000000	2023.000000	3.000000	5.000000	14987.000000	435291.000000

In [12]:
```
num_cols = df.select_dtypes(include="number").columns.tolist()
cat_cols = df.select_dtypes(exclude="number").columns.tolist()
print("Numerical:", num_cols)
print("Categorical:", cat_cols)
```

```
Numerical: ['Id', 'OverallQual', 'GrLivArea', 'GarageCars', 'TotalBsmtSF', 'YearBuilt', 'FullBath', 'BedroomAbvGr', 'LotArea',
'SalePrice']
Categorical: []
```

Data Pre-processing

In [13]:
```
df = df.dropna()
df = df.drop_duplicates()
df.shape
```

Out[13]:
```
(300, 10)
```

In [14]:
```
df = df.drop(columns="Id")
```

In [15]:
```
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

In [16]:
```
outliers = df[((df < lower_bound) | (df > upper_bound)).any(axis=1)]
outliers
```

Out[16]:

	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	BedroomAbvGr	LotArea	SalePrice

In [17]:
```
df = df[((df >= lower_bound) & (df <= upper_bound)).all(axis=1)]
df.shape
```

Out[17]:
```
(300, 9)
```

In [18]:
```
if cat_cols:
    df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
df.head()
```

Out[18]:

	OverallQual	GrLivArea	GarageCars	TotalBsmtSF	YearBuilt	FullBath	BedroomAbvGr	LotArea	SalePrice
0	7	1560	0	1658	1969	2	1	8059	177106
1	4	2827	2	1319	2012	3	4	13530	301044
2	8	3920	0	841	2010	1	4	9010	360609
3	5	3044	0	1058	1998	0	4	13207	240556
4	7	801	1	2428	2020	0	1	9117	193656

Feature Selection

In [19]:
```
corr = df.corr()["SalePrice"].drop("SalePrice").sort_values(ascending=False)
corr
```

Out[19]:
```
GrLivArea       0.745095
OverallQual     0.352899
GarageCars      0.351321
FullBath        0.273731
TotalBsmtSF     0.250178
YearBuilt       0.222142
BedroomAbvGr    0.014113
LotArea        -0.038982
Name: SalePrice, dtype: float64
```

In [20]:
```
features = corr[corr.abs() > 0.1].index.tolist()
features
```

Out[20]:
```
['GrLivArea',
 'OverallQual',
 'GarageCars',
 'FullBath',
 'TotalBsmtSF',
 'YearBuilt']
```

Linear Regression

In [21]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import r2_score
```

In [22]:
```
X = df[features]
y = df["SalePrice"]
```

In [23]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape)
```

```
(240, 6) (60, 6)
```

In [24]:
```
model = LinearRegression()
model.fit(X_train, y_train)
```

Out[24]:

LinearRegression

▸ Parameters

▸ Fitted attributes

In [25]:
```
pd.DataFrame({"Feature": features, "Coefficient": model.coef_})
```

Out[25]:

	Feature	Coefficient
0	GrLivArea	54.346202
1	OverallQual	11594.499253
2	GarageCars	19595.738362
3	FullBath	11210.048497
4	TotalBsmtSF	24.848906
5	YearBuilt	728.304038

In [26]:
```
model.intercept_
```

Out[26]:
```
np.float64(-1460478.4188176778)
```

Prediction

In [27]:
```
y_pred = model.predict(X_test)
```

Evaluation

In [28]:
```
r2_score(y_test, y_pred)
```

Out[28]:
```
0.9603226660500325
```

In [29]:
```
comparison = pd.DataFrame({"Actual": y_test.values, "Predicted": y_pred.round(2), "Difference": (y_test.values -
y_pred).round(2)})
comparison.head(15)
```

Out[29]:

	Actual	Predicted	Difference
0	224163	244135.51	-19972.51
1	203881	201401.62	2479.38
2	380236	391119.09	-10883.09
3	211664	228050.19	-16386.19
4	122139	141430.33	-19291.33
5	180067	180306.68	-239.68
6	275810	271650.13	4159.87
7	209025	201426.38	7598.62
8	213952	216255.99	-2303.99
9	154885	182613.77	-27728.77
10	395010	394811.93	198.07
11	253397	282969.36	-29572.36
12	115204	121874.47	-6670.47
13	189831	177858.04	11972.96
14	300918	290895.97	10022.03

In [ ]: